

Differenze tra script pilot e batch v1

01_detect_components

Valutazione generale

Per lo script `01_detect_components` **non sono state introdotte modifiche sostanziali alla logica di elaborazione**. La pipeline del passo 01 rimane invariata tra versione pilot e batch v1.

Cosa rimane uguale

La versione batch v1 mantiene invariati i passaggi principali:

1. caricamento del modello YOLO;
2. lettura del file YAML con i metadati delle classi;
3. costruzione di:
 - `class_meta`
 - `detect_class_ids`
 - `terminal_class_ids`
 - `masking_class_ids`
4. esecuzione dell'inferenza con gli stessi parametri principali (`imgsz` , `conf` , `iou`);
5. salvataggio di un JSON per immagine con:
 - metadati immagine
 - classi rilevabili
 - classi usate per terminali
 - classi usate per masking
 - lista dei componenti rilevati
6. salvataggio dell'immagine debug con bounding box e label. `:contentReference[oaicite:2]{index=2}`

Differenze effettive

Le differenze introdotte nel batch v1 sono solo di **organizzazione operativa**, non di logica algoritmica:

- è stata semplificata la gestione dell'input: la versione pilot supportava sia modalità `single` sia modalità `folder` , mentre la batch v1 lavora direttamente in modalità batch su una cartella di immagini;

- il resto del comportamento dello script è rimasto sostanzialmente invariato.

Sintesi

Il passo 01 nella batch v1 è quindi da considerare una **trasposizione del pilot al caso batch**, senza modifiche concettuali alla detection o alla struttura dei risultati.

02_assign_instances

Valutazione generale

Anche per lo script `02_assign_instances` **non ci sono modifiche sostanziali alla logica principale**.
L'algoritmo di assegnazione delle istanze rimane lo stesso tra pilot e batch v1.

Cosa rimane uguale

La procedura resta invariata:

1. lettura dei componenti dal JSON del passo 01;
2. raggruppamento per `class_id` ;
3. ordinamento dei componenti in base a `SORT_ORDER` ;
4. assegnazione degli `instance_id` nel formato `<class_id>.<indice>` ;
5. riordinamento finale dei componenti;
6. salvataggio del JSON aggiornato;
7. generazione dell'immagine debug con gli `instance_id` .

Differenze effettive

Nel batch v1 si osserva una sola piccola estensione utile del contenuto del JSON:

- viene aggiunto esplicitamente il campo `n_components` , pari al numero totale di componenti assegnati nell'immagine.

Oltre a questo, la logica di:

- calcolo del centro del bounding box,
- ordinamento `xy` o `yx` ,
- assegnazione delle istanze,
- disegno del debug

rimane la stessa della versione pilot.

Sintesi

Il passo 02 nella batch v1 è quindi anch'esso una versione sostanzialmente equivalente al pilot, con una piccola aggiunta informativa (`n_components`) ma senza cambiamenti algoritmici rilevanti.

03_estimate_terminals

Valutazione generale

Il passo 03 è quello in cui la pipeline batch v1 si discosta davvero dal pilot.

Nel pilot la stima dei terminali era basata quasi interamente sulla **geometria del bounding box** e sul contenuto dichiarato nel file YAML; nella batch v1, invece, la stima diventa **image-aware**, cioè usa anche informazione locale estratta direttamente dall'immagine binarizzata del diagramma.

In altre parole:

- **pilot**: terminali stimati soprattutto in base a regole statiche e aspect ratio;
- **batch v1**: terminali stimati combinando regole del metadata e analisi locale dei pixel vicino ai lati del simbolo.

Logica del pilot

Nel pilot erano presenti solo due modalità principali di definizione dei terminali:

1. **fixed**

Le posizioni dei terminali venivano lette direttamente dal metadata.

2. **auto_by_aspect_ratio**

L'orientazione del simbolo veniva dedotta dal rapporto larghezza/altezza del bounding box e, in base a questa orientazione, veniva scelta la configurazione dei terminali.

Questa impostazione era semplice e pulita, ma aveva un limite importante:

assumeva che il bounding box fosse un indicatore affidabile della direzione di connessione del

simbolo.

Per molti simboli questa ipotesi non è sempre vera. In particolare:

- alcuni simboli hanno bbox poco informativi;
- alcuni simboli hanno contatti reali non perfettamente allineati al centro del bbox;
- classi diverse con lo stesso numero di terminali non hanno necessariamente la stessa geometria di connessione.

Limiti emersi nel pilot

Durante il passaggio al batch sono emersi diversi casi in cui il pilot non era abbastanza robusto:

- **capacitor** con bbox ambiguo o disturbato da elementi vicini;
- **switch** aperti, in cui il bbox può risultare fuorviante rispetto alla direzione effettiva dei contatti;
- casi in cui il terminale reale è riconoscibile meglio guardando il **filo vicino al simbolo** che non la sola forma del bbox;
- casi in cui la classe `Terminal` non si comporta sempre come oggetto rigidamente monoconnesso.

Per questo motivo il passo 03 è stato ripensato in modo più robusto.

Modifiche introdotte nella batch v1

1. Passaggio da una stima “bbox-only” a una stima guidata anche dall’immagine

La modifica più importante è l'introduzione di una rappresentazione binaria del foreground tramite:

- conversione in grayscale;
- sogliatura Otsu inversa;
- uso della binarizzazione come base per analizzare i lati del simbolo.

Questo significa che la stima dei terminali non dipende più solo da:

- dimensioni del bbox,
- orientazione attesa nel metadata,

ma anche da **quanto foreground è presente localmente vicino ai lati del componente**.

2. Introduzione di nuove strategie di stima dei terminali

Nel pilot le strategie erano limitate.

Nella batch v1 il set di strategie gestite cresce in modo significativo. Oltre a `fixed` e `auto_by_aspect_ratio`, compaiono nuove modalità:

- `one_terminal_by_orientation`
- `two_terminal_by_connection_axis`
- `two_terminal_capacitor`
- `two_terminal_switch`
- `terminal_auto_one_or_two`

Questa estensione è importante perché separa casi che nel pilot venivano trattati in modo troppo uniforme.

3. Riconoscimento del lato realmente connesso per componenti a un terminale

Per i componenti a un terminale, la batch v1 introduce una logica che misura il foreground vicino ai quattro lati del bbox e prova a capire **da quale lato entra il collegamento**.

Questo avviene tramite funzioni come `detect_connected_side(...)` e `resolve_one_terminal_orientation(...)`.

Nel pilot questa informazione non veniva stimata dall'immagine: si passava direttamente dalla regola dichiarata nel metadata alla posizione del terminale.

Il vantaggio della nuova logica è che l'orientazione dei simboli monoconnessi non è più solo "dichiarata", ma può essere **inferita localmente dal contatto effettivo col wire**.

4. Nuova stima dell'asse di connessione per componenti a due terminali

Nel pilot, per i componenti a due terminali in modalità automatica, la scelta tra verticale e orizzontale dipendeva soprattutto dall'**aspect ratio del bbox**.

Nella batch v1, invece, l'orientazione può essere stimata misurando il foreground in prossimità di:

- lato alto o basso

- lato sinistro o destro

e confrontando i punteggi dei due assi.

Per **foreground** si intende l'insieme dei pixel che appartengono agli elementi grafici del diagramma, come simboli, fili e terminali, in contrapposizione al background, che rappresenta lo sfondo dell'immagine, ad esempio se vicino al lato destro di un componente trovi molti pixel di foreground, vuol dire che lì probabilmente passa un collegamento reale.

Questa logica è implementata tramite:

- `get_side_scores(...)`
- `get_local_terminal_probe_scores_center(...)`
- `_decide_axis_from_scores(...)`
- `detect_two_terminal_orientation_generic(...)`

Il risultato è che l'asse di connessione viene dedotto non solo dalla forma del simbolo, ma dalla **presenza locale di segnale grafico** compatibile con una connessione reale.

5. Introduzione di probe locali sui terminali candidati

Questa è una delle differenze più importanti dal punto di vista metodologico.

Nel batch v1 non si analizzano più solo bande laterali ampie, ma anche **probe locali stretti** centrati sui terminali candidati:

- top
- bottom
- left
- right

Questi probe sono delle finestre locali con cui il metodo campiona l'immagine e servono a capire quale coppia di lati è più compatibile con la connessione reale del simbolo.

Questa scelta rende il sistema molto più robusto nei casi in cui:

- il bbox è rumoroso;
- ci sono scritte o elementi vicini;
- il simbolo è sottile o sbilanciato;
- il contatto reale è localizzato in una zona specifica del lato.

6. Separazione esplicita tra logica generica, logica per capacitor e logica per switch

Questa è probabilmente la modifica concettualmente più importante del passo 03.

Nel pilot simboli diversi ma tutti “a due terminali” venivano trattati con una logica abbastanza uniforme.

Nel batch v1, invece, si riconosce che **non tutti i simboli a due terminali hanno la stessa geometria elettrica**, e quindi vengono trattati separatamente.

In particolare:

- `detect_two_terminal_orientation_capacitor(...)`
usa probe centrati adatti ai capacitor e fallback coerenti con quel tipo di simbolo;
- `detect_two_terminal_orientation_switch(...)`
usa probe multi-anchor, più adatti ai contatti di uno switch aperto;
- `detect_two_terminal_orientation_generic(...)`
gestisce gli altri casi due-terminal con una logica più generale.

Questa separazione è stata introdotta perché una singola euristica comune portava a continui compromessi: migliorare uno switch rischiava di peggiorare un capacitor, e viceversa. La batch v1 supera questo limite introducendo una **modellazione per famiglie di simboli**.

7. Gestione specifica dello switch tramite probe multi-anchor

Per gli switch, i contatti reali possono non essere centrati nel bbox.

Per questo la batch v1 introduce una scansione su **più anchor** lungo i lati del componente, invece di usare solo il centro geometrico. Questa logica compare in

`get_local_terminal_probe_scores_multi_anchor(...)` ed è usata dalla strategia specifica per switch.

Questa modifica è fondamentale per il caso di `e1e43`, dove lo switch `25.1` viene riconosciuto come **horizontal** e i terminali vengono stimati sui lati corretti, anche se il bbox da solo non sarebbe stato affidabile.

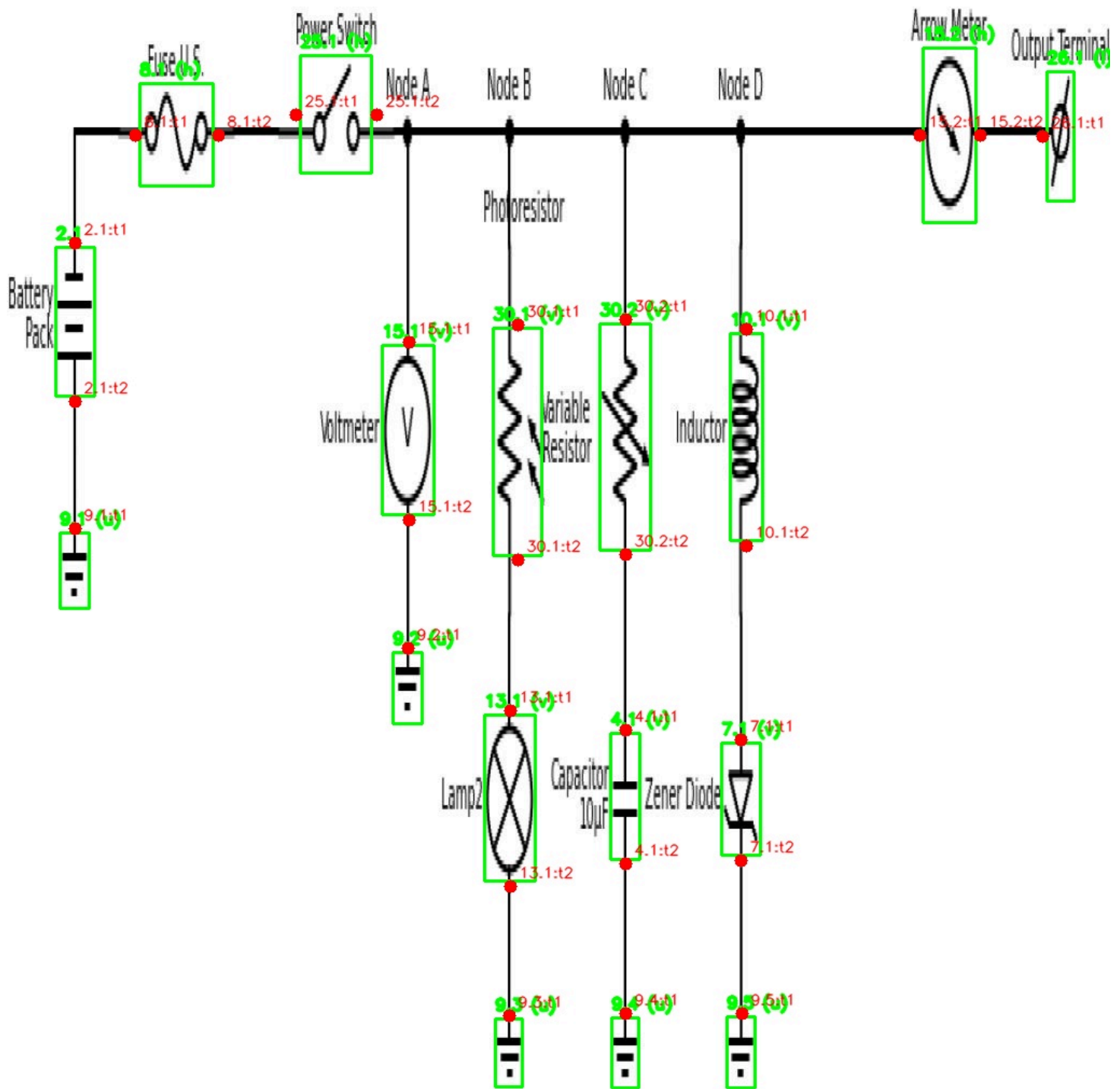


Figura 3.1 — Caso switch (e1e43 , 25.1).

Esempio di simbolo per cui il bounding box da solo non è sufficiente a stimare correttamente la direzione di connessione. In batch v1 il componente viene trattato con una strategia dedicata basata su **probe multi-anchor** lungo i lati del bbox, così da intercettare contatti laterali non perfettamente centrati. L'immagine mostra il bounding box del componente e i terminali stimati sui lati corretti.

8. Gestione specifica del capacitor tramite probe centrati

Per i capacitor, al contrario, la geometria di connessione è spesso ben rappresentata da probe centrati sull'asse del simbolo.

La batch v1 introduce quindi una logica dedicata che privilegia proprio questo comportamento, evitando che il capacitor venga trattato come uno switch o come un generico two-terminal.

Questo è visibile per esempio in:

- `ele43` , dove il capacitor `4.1` viene mantenuto correttamente **vertical**;
- `spider` , dove il capacitor `4.2` in alto viene correttamente riconosciuto **horizontal**.

Questi esempi mostrano bene perché la separazione capacitor/switch non è solo un dettaglio implementativo, ma una vera modifica strutturale della logica del passo 03.

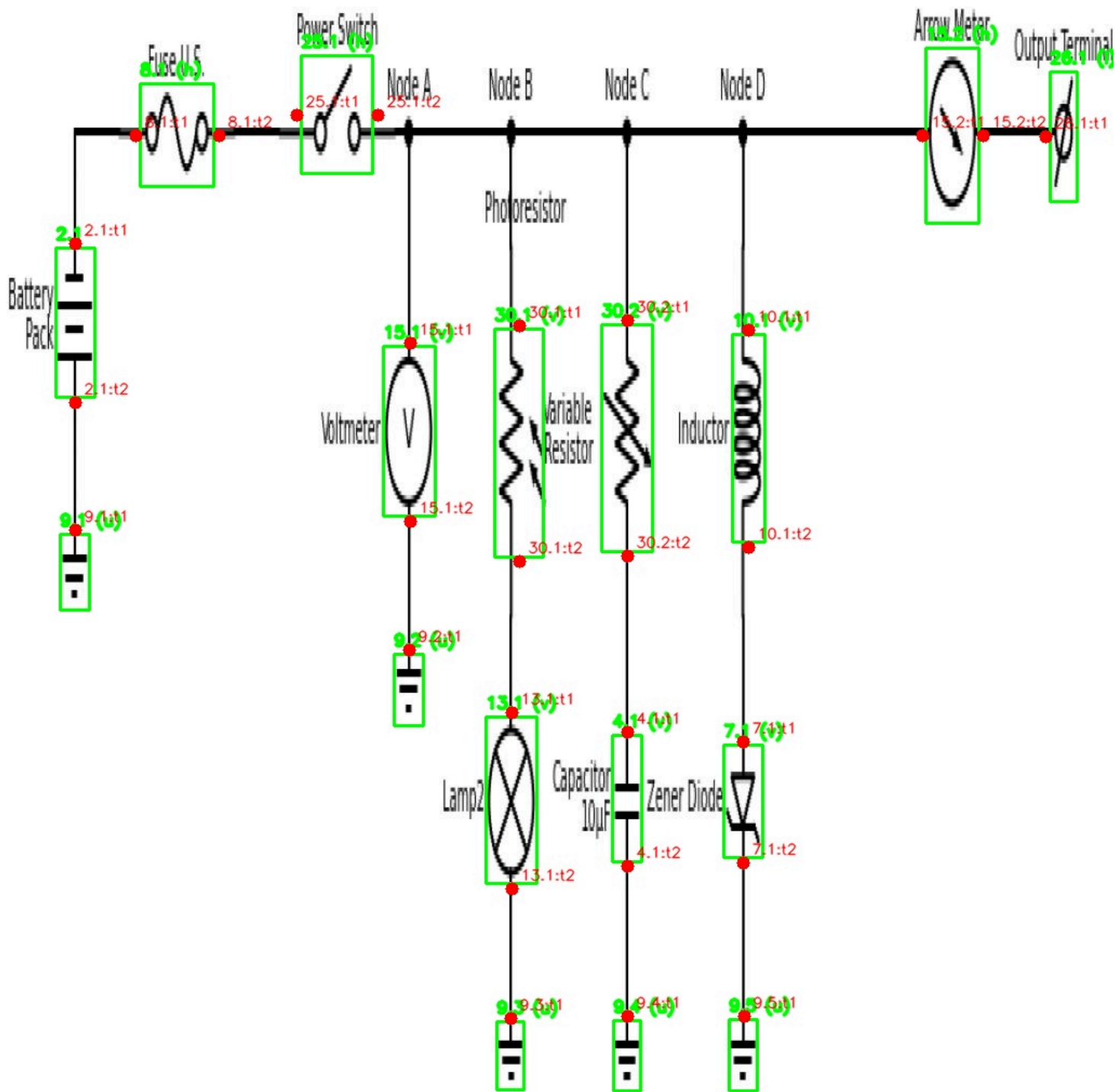


Figura 3.2 — Caso capacitor verticale (e1e43 , 4.1).

Questo esempio mostra perché il capacitor non può essere trattato con la stessa euristica dello switch. In batch v1 viene usata una logica specifica per i capacitor, basata su **probe centrati** sull'asse del simbolo, che consente di mantenere correttamente l'orientazione verticale e di collocare i terminali in corrispondenza dei lati superiore e inferiore.

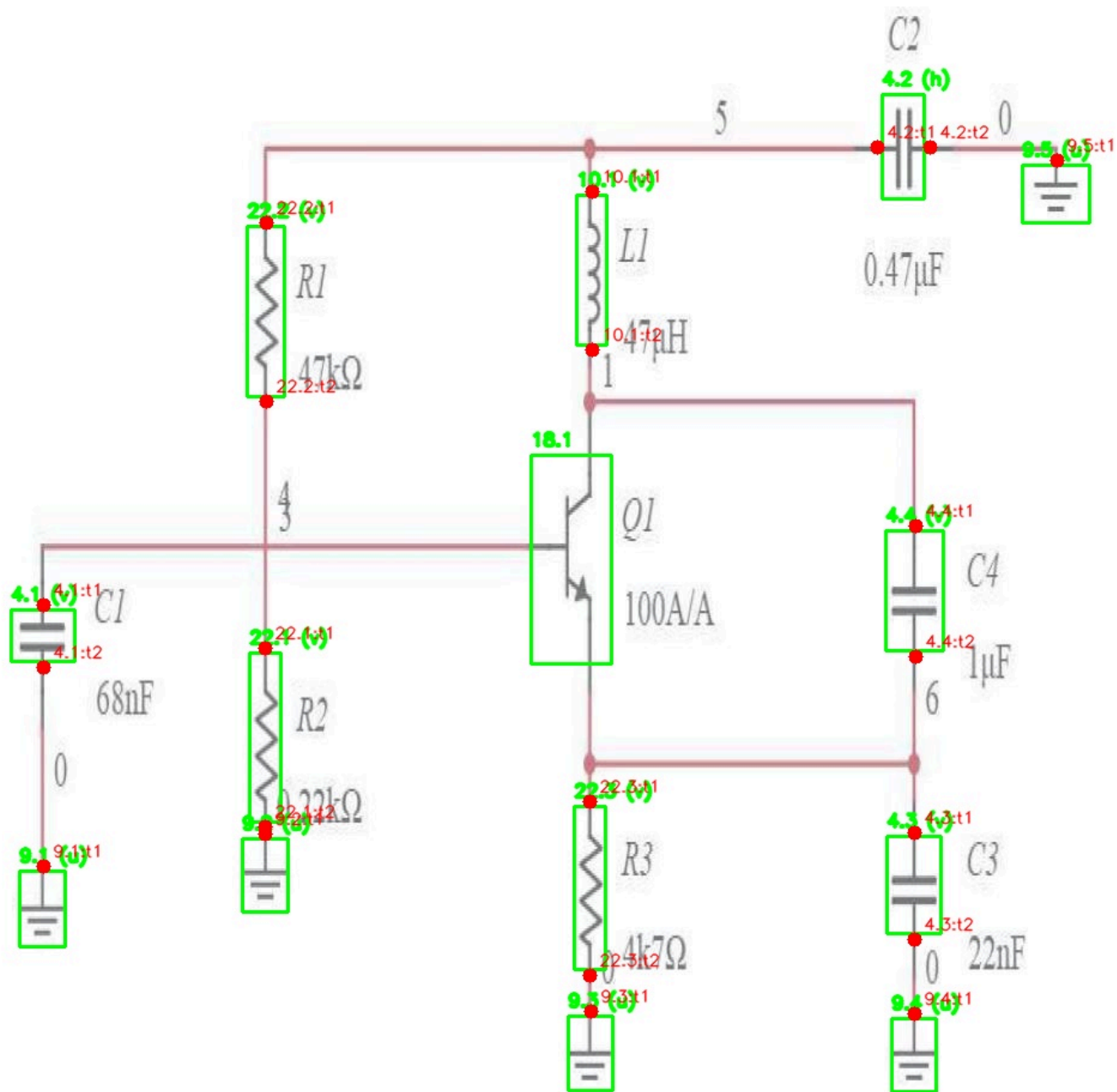


Figura 3.3 — Caso capacitor orizzontale (spider , 4.2).

L'esempio evidenzia che la nuova logica introdotta nel batch v1 non favorisce solo simboli verticali, ma consente di riconoscere correttamente anche un capacitor disposto orizzontalmente. I terminali vengono infatti stimati sui lati sinistro e destro del simbolo, coerentemente con la reale direzione di connessione.

9. Supporto a componenti “Terminal” con cardinalità variabile

Nel batch v1 compare anche la strategia `terminal_auto_one_or_two`, che permette di trattare la classe `Terminal` in modo più flessibile.

Questo è importante perché nei dati reali la classe non si comporta sempre come rigidamente monoconnessa: in alcuni casi ha un solo terminale, in altri ne ha due.

Di conseguenza, il batch v1 non impone più una cardinalità fissa basata solo sull'assunzione iniziale della classe, ma lascia che la struttura finale dipenda dalla stima effettiva dei terminali.

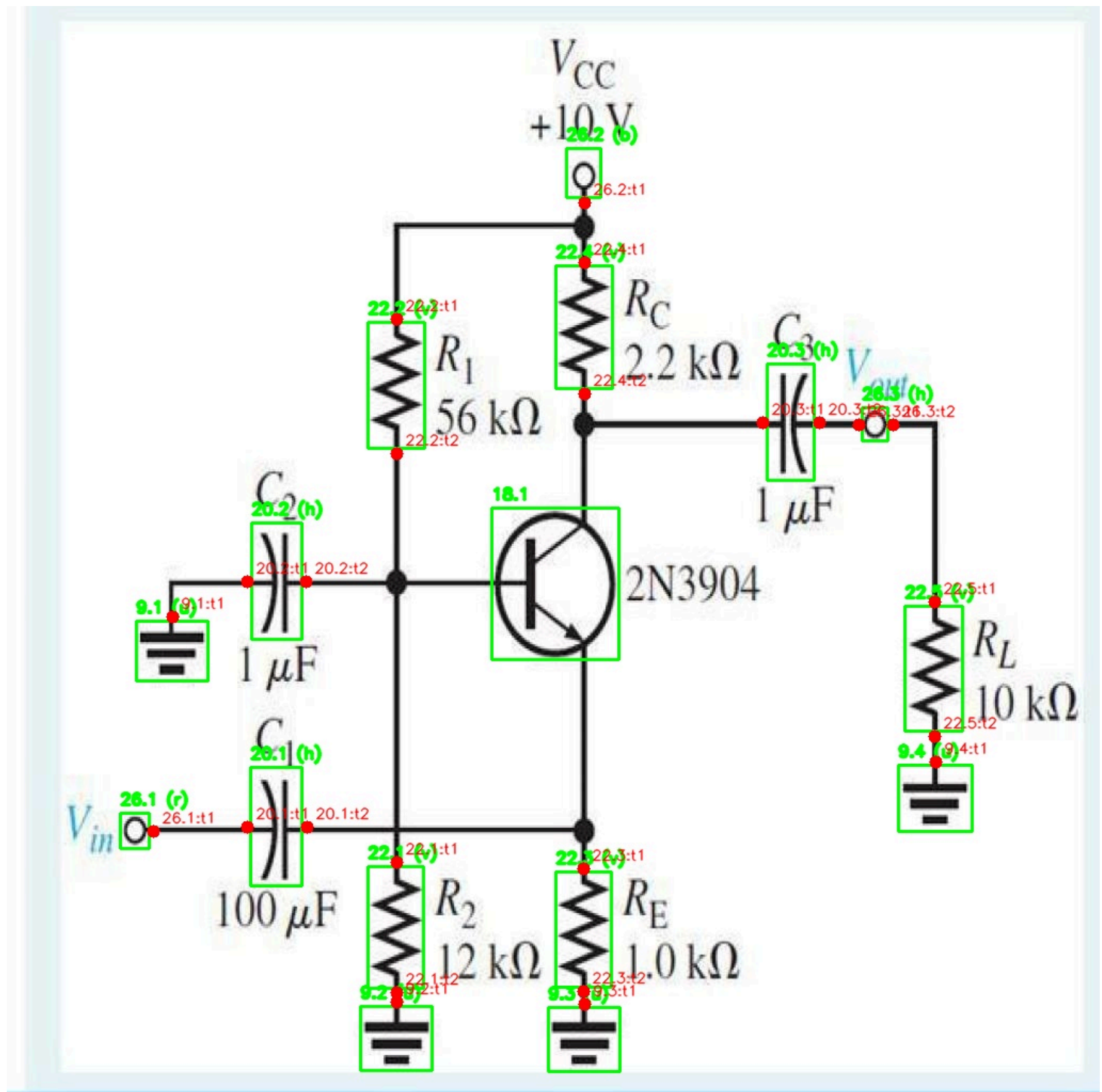


Figura 3.4 — Caso `Terminal` con due terminali (1084 , 26.3).

Questo caso documenta una differenza importante emersa durante il passaggio al batch v1: la classe `Terminal` non si comporta sempre come oggetto rigidamente monoconnesso. In alcuni diagrammi può presentare due punti di connessione, e per questo la pipeline batch v1 non impone più una cardinalità fissa, ma lascia che il numero di terminali venga determinato dalla stima effettiva sul componente.

10. Spostamento del terminale leggermente all'esterno del bbox

Nel pilot il terminale veniva posizionato esattamente sul bordo del bounding box.

Nel batch v1 viene introdotto un piccolo `TERMINAL_OUTWARD_OFFSET`, quindi il punto terminale viene spostato leggermente **all'esterno** del bbox.

Questa scelta ha due vantaggi pratici:

- evita di collocare il terminale “dentro” il simbolo;
- rende più naturale il successivo aggancio ai wire nei passi successivi della pipeline.

11. Arricchimento del JSON di output

Nel pilot il passo 03 salvava principalmente:

- i terminali stimati;
- l'orientazione eventualmente dedotta.

Nel batch v1 l'output viene arricchito con nuove informazioni diagnostiche utili per i passi successivi e per il debug:

- `estimated_connection_side`
- `connection_side_scores`
- `estimated_orientation`
- terminali con posizione aggiornata e più coerente con il lato di connessione.

Questo rende il passo 03 non più solo un generatore di terminali, ma anche un punto di **diagnostica esplicita** sulla qualità e sulla motivazione della stima.

Sintesi della differenza concettuale

La differenza principale tra pilot e batch v1 può essere riassunta così:

- **pilot**: il terminale è una proprietà quasi interamente derivata da metadata + bbox;
- **batch v1**: il terminale è una proprietà derivata da metadata + bbox + analisi locale del contenuto grafico vicino ai lati del simbolo.

Quindi il passo 03 della batch v1 non è una semplice rifinitura del pilot:

è una **evoluzione strutturale** che rende la stima dei terminali molto più aderente alla geometria reale dei diagrammi.

04_extract_wires

Valutazione generale

Nel passo 04 la struttura generale della pipeline rimane sostanzialmente invariata tra pilot e batch v1.

In entrambe le versioni lo script:

1. legge l'immagine originale;
2. maschera i componenti da rimuovere;
3. preserva una piccola zona attorno ai terminali stimati;
4. **binarizza l'immagine**: processo con cui l'immagine viene trasformata in una rappresentazione a due classi, foreground e background, così da isolare i tratti grafici rilevanti dal resto dell'immagine (wire e tratti 1, sfondo 0);
5. applica una **chiusura morfologica**: operazione applicata all'immagine binaria per chiudere piccole interruzioni e rendere più continui i tratti del wire;
6. rimuove opzionalmente piccoli componenti connessi;
7. produce lo **skeleton finale** dei wire: La **skeletonization** è un'operazione applicata all'immagine binaria che riduce i tratti del wire alla loro linea mediana, producendo una rappresentazione molto sottile della rete di connessioni, pur preservandone la struttura topologica.

La differenza sostanziale non riguarda quindi la sequenza dei passi, ma **il modo in cui vengono preservate le connessioni locali in prossimità dei terminali**.

Logica del pilot

Nel pilot la maschera dei componenti veniva costruita in due passaggi principali:

- disegno dei bounding box ristretti dei componenti da mascherare;
- riapertura locale della maschera in corrispondenza dei terminali tramite un **piccolo cerchio** centrato sul terminale stimato.

In pratica, ogni terminale proteggeva solo un intorno isotropo locale, senza tenere conto della direzione del contatto reale con il wire. Questa scelta era coerente con la pipeline pilot, in cui i terminali del passo 03 venivano collocati direttamente sul bordo del bounding box e risultavano, in media, abbastanza vicini al punto di contatto atteso.

Limite emerso nel passaggio al batch v1

Con la nuova versione del passo 03, i terminali non sono più semplicemente “punti sul bordo del bbox”, ma **anchor geometrici** più robusti, spesso posizionati leggermente all'esterno del componente e determinati usando anche informazione locale dell'immagine.

Questa evoluzione rende il passo 03 più corretto dal punto di vista topologico, ma introduce un effetto collaterale importante per il passo 04: **il terminale stimato non cade sempre esattamente sul pixel del wire**.

Di conseguenza, la preservazione tramite solo cerchio locale del pilot può diventare troppo fragile:

- se il terminale è leggermente fuori asse rispetto al cavo;
- se il contatto reale è allungato lungo una direzione;
- se il wire entra nel simbolo con una piccola discontinuità locale;
- se il simbolo è sottile o irregolare, come nei casi di switch o componenti verticali stretti.

Modifiche introdotte nella batch v1

1. La pipeline di estrazione rimane la stessa

La batch v1 mantiene invariati i blocchi principali del pilot:

- conversione in grayscale;
- applicazione della maschera componenti;

- closing morfologico;
- filtro opzionale per piccoli componenti connessi;
- skeletonization finale.

2. Separazione tra maschera base dei componenti e preservazione dei terminali

Nel pilot la funzione `build_component_mask(...)` costruiva direttamente la maschera finale, includendo sia i rettangoli dei componenti sia la riapertura circolare sui terminali.

Nella batch v1 questa logica viene resa più esplicita e modulare:

- `build_base_component_mask(...)` costruisce la maschera dei componenti;
- `carve_terminal_keep_zones(...)` riapre selettivamente le zone da preservare attorno ai terminali;
- `build_component_mask(...)` combina i due passaggi.

Questa scomposizione rende più chiaro il ruolo dei due contributi:

- **copertura del simbolo**
- **protezione locale del contatto col wire**

3. Passaggio da preservazione circolare a preservazione “direzionata”

Questa è la modifica più importante del passo 04.

Nel pilot ogni terminale proteggeva solo un **cerchio** di raggio fissato (`TERMINAL_KEEP_RADIUS`).

Nella batch v1, invece, ogni terminale preserva:

- un **cerchio locale** attorno al punto terminale;
- una **piccola capsula direzionata** ottenuta come segmento spesso orientato lungo il lato stimato del terminale.

La direzione dipende da `relative_position` :

- `left / right` → segmento orizzontale;
- `top / bottom` → segmento verticale.

Questa scelta è esplicitata nella funzione `terminal_keep_segment(...)`, che costruisce una piccola regione orientata coerente con il lato di connessione del terminale.

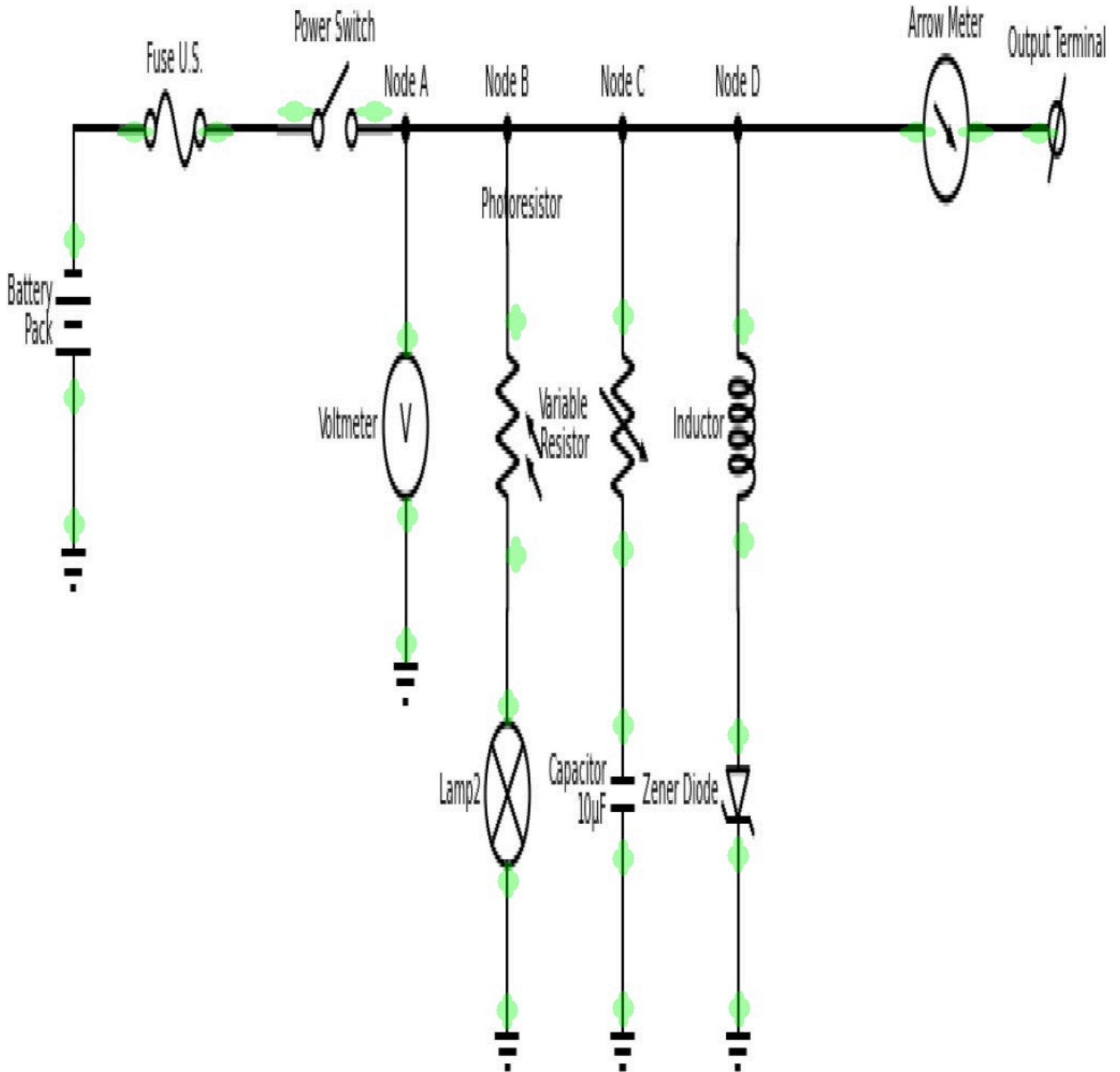


Figura 4.1 — Preservazione direzionata della zona terminale nel batch v1.

Nel passo 04 della batch v1 la maschera dei componenti non viene più riaperta solo con un piccolo cerchio centrato sul terminale, ma con una regione locale orientata lungo il lato di connessione stimato. Questa scelta rende più robusta la conservazione del tratto di wire adiacente al terminale anche quando il punto stimato non coincide perfettamente con il cavo.

4. Introduzione di una preservazione asimmetrica inward/outward

Nel pilot il cerchio locale era simmetrico in tutte le direzioni.

Nella batch v1 la capsula direzionata ha una struttura **asimmetrica**:

- una parte si estende **verso l'interno** del simbolo;
- una parte si estende **verso l'esterno**, cioè verso la zona in cui è atteso il wire.

Questo comportamento è controllato da parametri distinti:

- `TERMINAL_KEEP_INWARD_LEN`
- `TERMINAL_KEEP_OUTWARD_LEN`
- `TERMINAL_KEEP_LINE_THICKNESS`

La motivazione è pratica: il wire può non toccare esattamente il centro del terminale stimato, ma essere leggermente spostato lungo la direzione di connessione. La capsula orientata aumenta la probabilità di mantenere intatto quel tratto locale del cavo.

5. Adattamento esplicito alla nuova natura dei terminali del passo 03

Nel JSON della batch v1 il passo 04 dichiara esplicitamente che la nuova preservazione serve a tollerare **terminali stimati non perfettamente appoggiati sul cavo**.

Questo è il punto concettuale chiave dell'evoluzione del passo 04:

- il pilot assumeva terminali quasi “pixel-perfect”;
- la batch v1 assume terminali topologicamente corretti ma non necessariamente allineati al pixel del wire.

Di conseguenza, il passo 04 diventa un **ponte robusto** tra:

- terminali stimati dal passo 03;
- wire reali che verranno poi elaborati nei passi 05 e 06.

6. Introduzione di un debug esplicito per le zone preservate

Nel pilot era presente il debug della maschera dei componenti (`mask_debug`), ma non esisteva una visualizzazione separata e dedicata delle zone di preservazione attorno ai terminali.

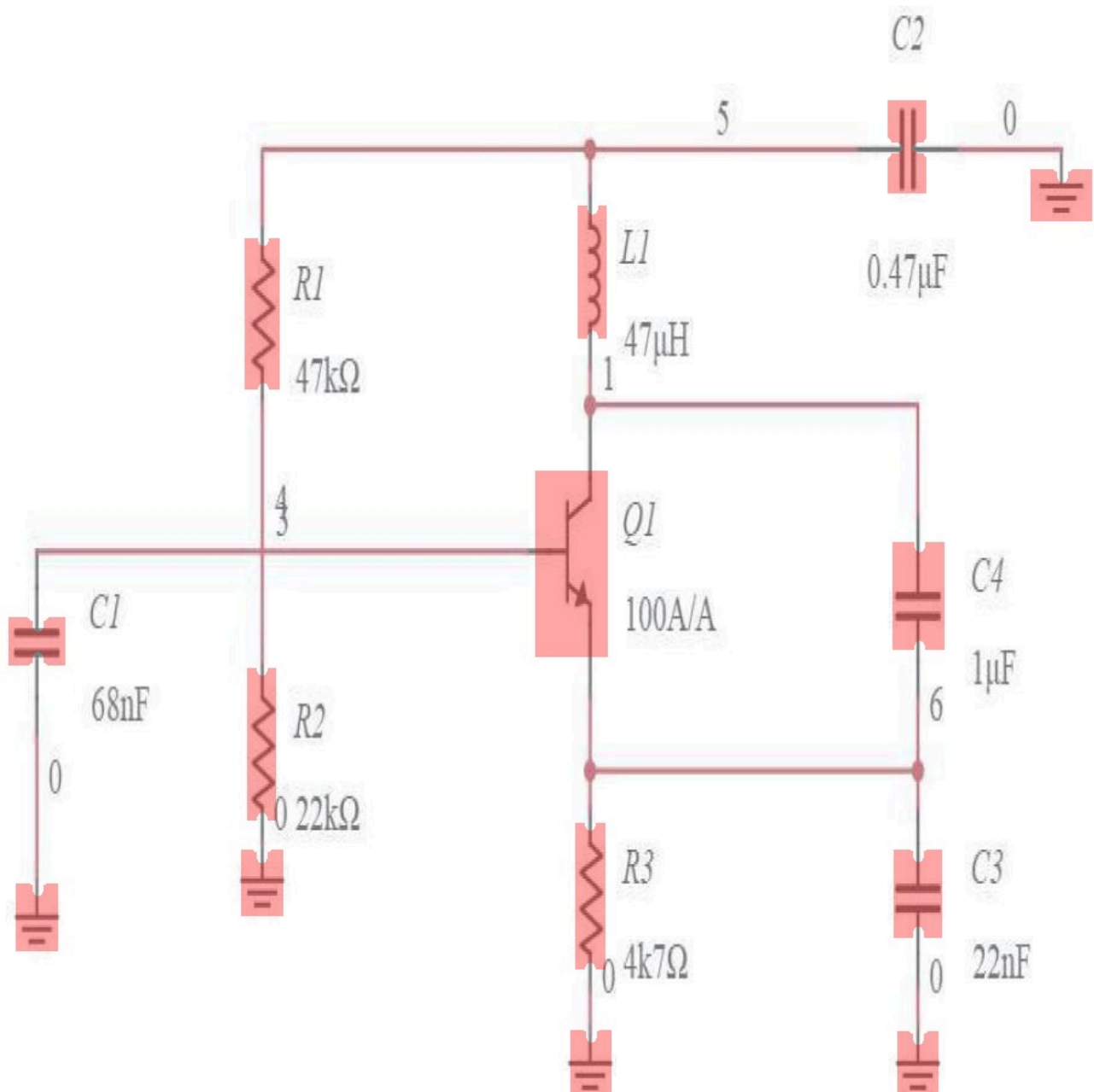
Nella batch v1 viene introdotto un output aggiuntivo:

- `terminal_keep_debug`

insieme alla funzione dedicata `save_terminal_keep_debug(...)`.

Questo output è molto utile perché rende immediatamente visibile:

- dove il componente viene effettivamente mascherato;
- dove invece la maschera viene riaperta per preservare il contatto elettrico locale.



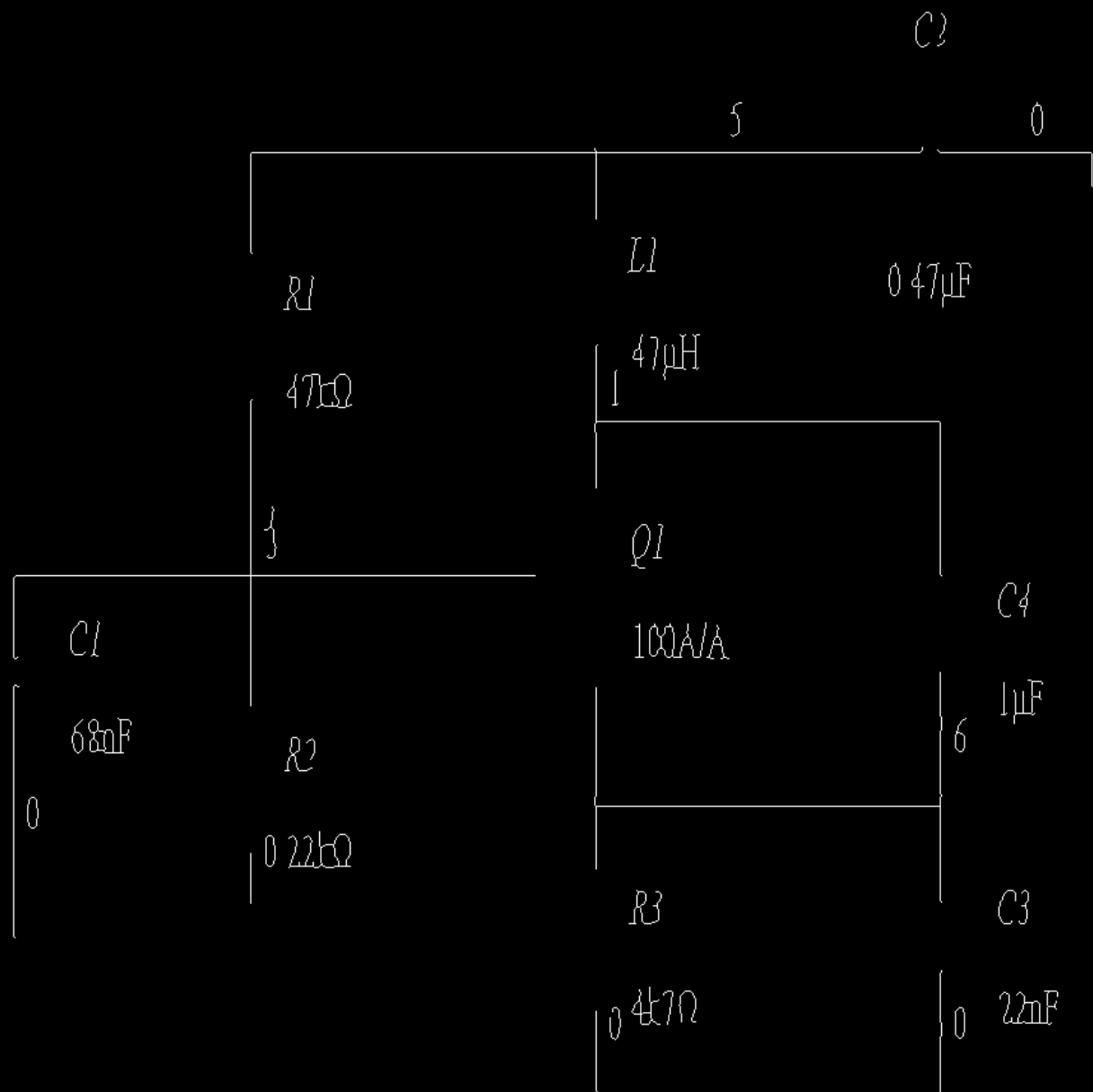


Figura 4.2 — Effetto della preservazione locale sull'estrazione dei wire.

Le immagini mostrano come la riapertura direzionata della maschera in prossimità dei terminali consenta di mantenere il tratto di collegamento locale tra simbolo e wire durante il masking dei componenti. Questo migliora la continuità del wire nelle immagini binarie e nello skeleton finale, riducendo il rischio che il contatto venga interrotto.

7. Il filtro dei piccoli componenti e la skeletonization restano invariati

Il filtro opzionale dei piccoli connected components resta sostanzialmente identico tra pilot e batch v1:

- stessa funzione basata su `connectedComponentsWithStats` ;
- stesso uso della soglia minima di area;
- stessa logica di conteggio tra componenti mantenuti e rimossi.

Anche la skeletonization finale rimane invariata:

- conversione dell'immagine filtrata in booleano;
- applicazione di `skeletonize` ;
- riconversione in immagine binaria `uint8` .

Questo conferma che la differenza centrale del passo 04 non riguarda il post-processing morfologico finale, ma la **fase di masking/preservazione locale**.

8. Il testo non viene ancora rimosso esplicitamente

In entrambe le versioni viene dichiarato esplicitamente che il testo non è ancora rimosso in modo dedicato, e che quindi le immagini binarie, chiuse, filtrate e skeleton possono contenere residui testuali.

Questa continuità è importante da documentare perché chiarisce che:

- il passo 04 batch v1 è più robusto sul contatto wire-terminal;
- ma non introduce ancora una pulizia esplicita delle scritte.

9. Arricchimento del blocco `wire_extraction` nel JSON

Nel pilot il blocco `wire_extraction` salvava principalmente:

- parametri di masking;
- parametri morfologici;
- informazioni sul filtro dei componenti piccoli;
- path dei file intermedi generati.

Nella batch v1 questo blocco viene reso più descrittivo e strutturato:

- le informazioni di preservazione dei terminali vengono raccolte nel sottoblocco `terminal_keep` ;
- viene aggiunto il path di `terminal_keep_debug` ;
- la nota esplicativa descrive il ruolo della preservazione direzionata.

Anche questo contribuisce a rendere il passo 04 più leggibile e meglio documentato.

Sintesi della differenza concettuale

La differenza fondamentale tra pilot e batch v1 nel passo 04 può essere riassunta così:

- **pilot**: il masking dei componenti preserva attorno ai terminali solo un piccolo intorno circolare;
- **batch v1**: il masking preserva attorno ai terminali una zona locale più robusta, composta da un cerchio e da una piccola regione orientata lungo il lato di connessione stimato.

:contentReference[oaicite:29]{index=29}

Questa modifica non cambia la pipeline nel suo insieme, ma cambia in modo sostanziale la sua robustezza:

il passo 04 della batch v1 è stato adattato per funzionare correttamente con terminali stimati in modo più realistico dal passo 03, senza richiedere che essi coincidano esattamente con il pixel del cavo.

05_build_nets

Valutazione generale

Nel passo 05 la struttura generale della pipeline rimane coerente tra pilot e batch v1.

In entrambe le versioni lo script:

1. legge lo skeleton prodotto dal passo 04;
2. calcola le connected components;
3. tratta ciascuna connected component come candidata net;
4. verifica quali terminali risultano vicini a quella componente;
5. filtra le candidate non significative;
6. rinumerava le net mantenute come `N1` , `N2` , `N3` , ...

La differenza principale non è quindi nella costruzione delle net in sé, ma nella fase di **associazione terminale** → **connected component**, che nella batch v1 viene resa molto più robusta e meglio

tracciabile.

Logica del pilot

Nel pilot l'associazione tra terminali e connected components dello skeleton avveniva tramite una logica semplice:

- per ogni terminale si prendeva una **piccola finestra quadrata** centrata sul punto terminale;
- si raccoglievano i label delle connected components presenti in quella finestra;
- da questi label si ricavava quali componenti dello skeleton erano candidate a diventare net.

Questa soluzione era efficace nei casi semplici, ma assumeva implicitamente che:

- il terminale stimato fosse già molto vicino al wire;
- un intorno quadrato isotropo fosse sufficiente a catturare il collegamento corretto.

Limiti emersi nel pilot

Con la pipeline batch v1, il passo 03 produce terminali più realistici dal punto di vista topologico, ma non sempre perfettamente coincidenti con il pixel del wire.

Inoltre, il passo 04 preserva il contatto locale in modo più robusto ma non garantisce che il punto terminale cada esattamente sullo skeleton.

Di conseguenza, nel passo 05 il solo approccio “finestra quadrata attorno al terminale” del pilot può risultare troppo debole in casi come:

- terminali laterali di uno **switch**;
- componenti allungati o sottili;
- terminali verticali/orizzontali leggermente fuori asse rispetto al wire;

Modifiche introdotte nella batch v1

1. La costruzione delle candidate nets rimane invariata

La batch v1 mantiene invariata la logica generale di costruzione delle net:

- `connectedComponentsWithStats(...)` sullo `skeleton`;
- costruzione delle candidate con:
 - `source_label`
 - `pixel_count`
 - `bbox`
 - `connected_terminal_ids`
 - `n_connected_terminals`
- filtraggio in base a:
 - numero minimo di pixel
 - numero minimo di terminali connessi;
- rinumerazione ordinata delle net finali.

Questa continuità è importante: anche nel batch v1 il passo 05 continua a essere un passo di **costruzione e filtraggio di candidate nets** a partire dallo `skeleton`.

2. Sostituzione della sola finestra quadrata con una ricerca direzionale

Questa è la modifica più importante del passo 05.

Nel pilot la funzione `terminal_to_candidate_labels(...)` usava solo una finestra quadrata di raggio fisso attorno al terminale.

Nella batch v1, invece, l'associazione terminale → connected component avviene in due fasi:

1. **ricerca direzionale primaria**, coerente con `relative_position` del terminale;
2. **fallback quadrato locale**, usato solo se la ricerca direzionale non trova nulla.

In pratica:

- terminali `left/right` privilegiano una finestra allungata orizzontalmente;
- terminali `top/bottom` privilegiano una finestra allungata verticalmente.

Questa scelta rende l'associazione più coerente con la geometria del contatto atteso.

3. Introduzione di una nozione di `primary_label`

Nel pilot la funzione restituiva principalmente:

- mappa `label_to_terminal_ids`
- mappa `terminal_to_labels` con tutti i label vicini al terminale.

Nella batch v1 viene introdotto un concetto più forte:

- per ogni terminale si prova a determinare un **primary_label**, cioè la connected component principale a cui il terminale sembra appartenere.

Questo `primary_label` viene ricavato cercando, nella finestra locale, il **pixel etichettato più vicino** al terminale.

Di conseguenza, il passo 05 non si limita più a dire “questi label sono vicini”, ma fornisce già una **preferenza locale esplicita** per il match terminale → net.

Questa informazione sarà poi molto utile nel passo 06.

4. Introduzione dello snap locale al pixel etichettato più vicino

Nel pilot non veniva esplicitamente salvato il punto di aggancio locale del terminale alla connected component.

Nella batch v1 viene introdotta la funzione `find_nearest_labeled_pixel(...)`, che per ogni terminale cerca:

- il pixel di skeleton etichettato più vicino nella finestra locale;
- la distanza di snap;
- il label corrispondente.

Questo produce, per ogni terminale:

- `primary_label`
- `snap_point`
- `snap_distance`
- `match_mode` / modalità di ricerca usata.

È una differenza importante, perché rende il passo 05 non più solo un costruttore di net, ma anche un produttore di **vincoli locali di associazione**.

5. Tracciamento esplicito della modalità di match

Nel pilot non c'era una distinzione esplicita tra diversi tipi di ricerca locale per il terminale.

Nella batch v1 viene invece salvato per ogni terminale se il match è stato ottenuto con:

- `directional`
- `square_fallback`

Questo è utile perché consente di distinguere:

- match ottenuti in modo coerente con la direzione del terminale;
- match recuperati con un fallback più permissivo.

Dal punto di vista documentale, questa è una piccola ma importante aggiunta di trasparenza.

6. Arricchimento del debug visivo

Nel pilot le immagini debug principali erano:

- `net_map`
- `overlay`

Nella batch v1 viene aggiunta anche:

- `terminal_debug`

Questa nuova visualizzazione mostra:

- il punto terminale stimato;
- il punto di snap sullo skeleton;
- il collegamento tra i due;
- la net finale associata.

È una differenza utile perché rende molto più leggibile il comportamento dello script nei casi dubbi.

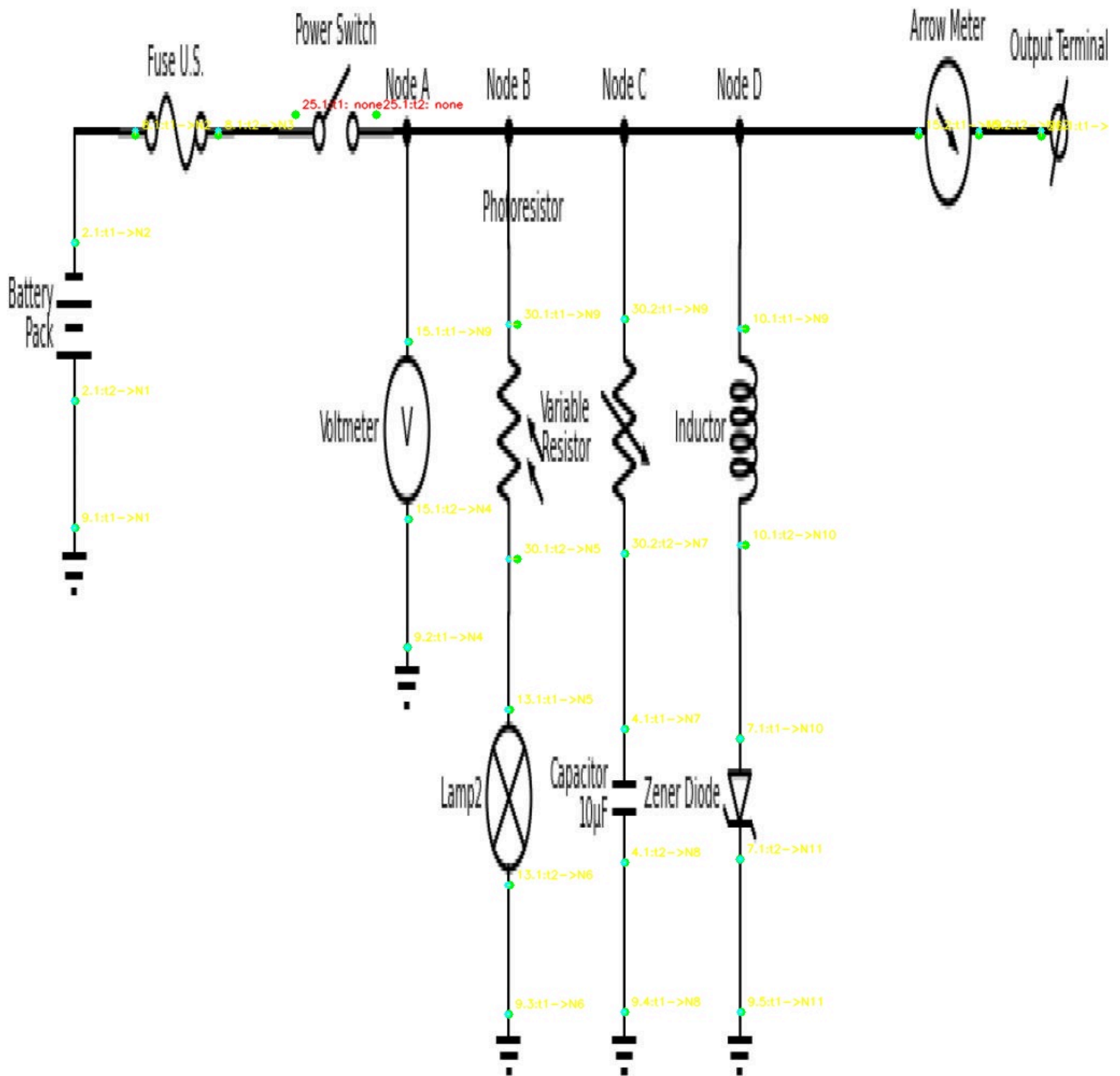


Figura 5.1 — Visualizzazione del matching locale tra terminali e connected components dello skeleton.

La batch v1 introduce un debug dedicato che mostra, per ciascun terminale, il punto stimato, il punto di snap sullo skeleton e la net associata. Questa visualizzazione rende più trasparente il comportamento dello script e permette di distinguere i casi in cui il collegamento è ottenuto direttamente con ricerca direzionale da quelli recuperati con fallback locale.

7. Introduzione di diagnostica sui componenti rifiutati

Nel pilot le candidate non mantenute venivano semplicemente escluse dal risultato finale.

Nella batch v1, invece, i componenti scartati vengono mantenuti nella struttura `rejected_candidates` insieme alle rispettive motivazioni, ad esempio:

- `too_few_pixels`
- `no_connected_terminals`

Questo consente di capire meglio se:

- lo skeleton è troppo frammentato;
- ci sono ancora residui testuali o piccoli artefatti;
- alcune componenti non sono state promosse a net per motivi sensati.

8. Estensione del blocco `net_building` nel JSON

Nel pilot il blocco `net_building` conteneva soprattutto:

- i parametri di soglia;
- il numero di candidate;
- il numero di net tenute/scartate;
- la mappa `terminal_to_candidate_labels` ;
- i path dei file debug.

Nella batch v1 questo blocco viene esteso con nuove informazioni:

- `notes`
- `terminal_search_radius`
- `terminal_directional_halfspan`
- `n_connected_components_total`
- `n_terminals`
- `n_terminals_with_primary_label`
- `n_terminals_unmatched`
- `rejected_candidates`
- `terminal_debug_path`

Questa evoluzione rende il passo 05 molto più informativo e più utile per il debugging dell'intera pipeline.

Sintesi della differenza concettuale

La differenza principale tra pilot e batch v1 nel passo 05 può essere riassunta così:

- **pilot:** le net vengono costruite a partire dalle connected components dello skeleton, e i terminali vengono associati a esse tramite semplice prossimità locale in una finestra quadrata;
- **batch v1:** la costruzione delle net rimane la stessa, ma l'associazione terminale → connected component viene resa più robusta grazie a ricerca direzionale, snap locale al pixel etichettato più vicino e diagnostica esplicita del match.

Di conseguenza, il passo 05 della batch v1 non cambia la definizione di net, ma migliora in modo significativo la qualità e la leggibilità della fase che collega i terminali stimati del passo 03 alle connected components prodotte dal passo 04.

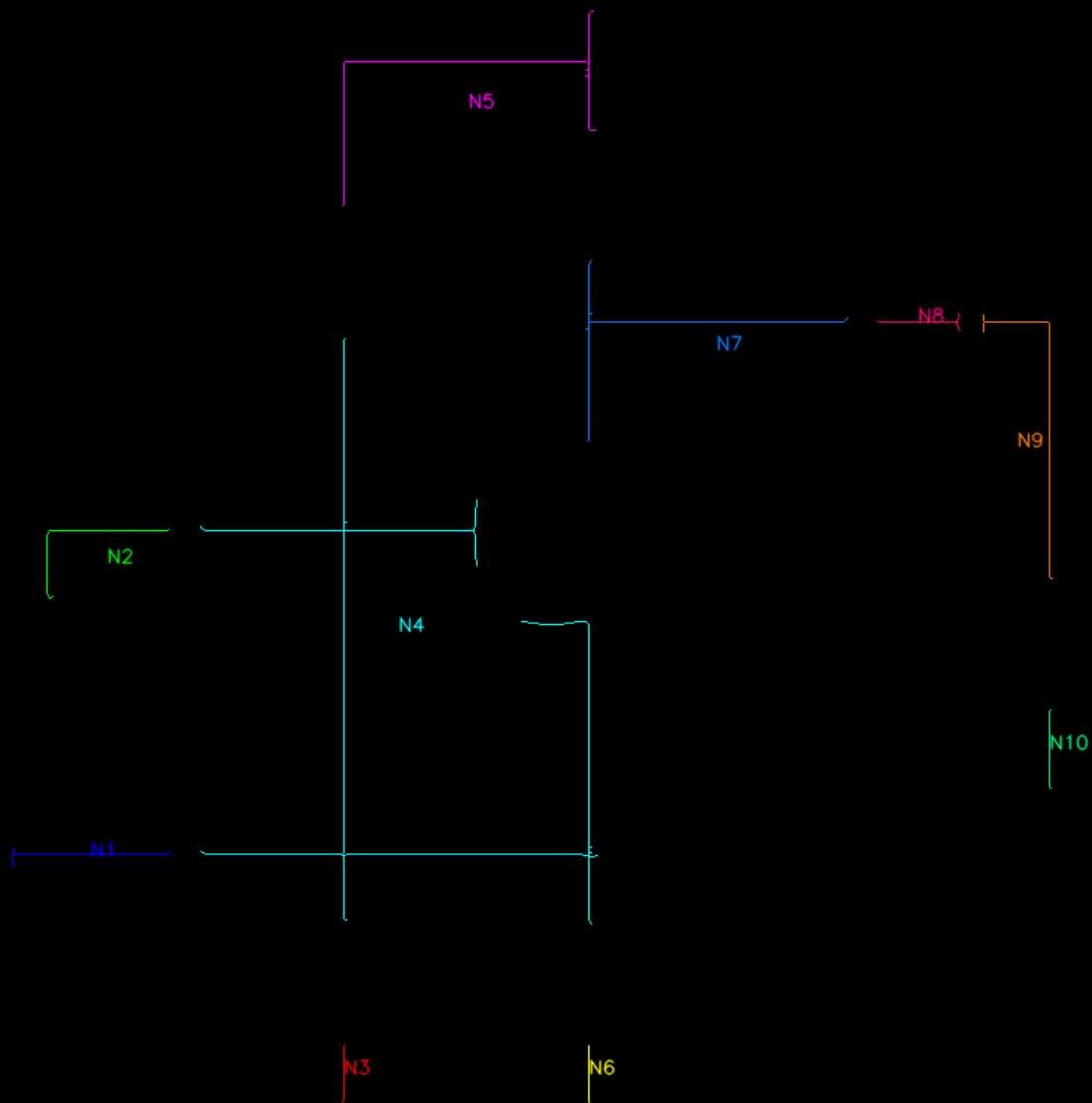


Figura 5.2 — Mappa finale delle net costruite a partire dallo skeleton del passo 04.

L'immagine mostra le connected components dello skeleton che, dopo il filtraggio, vengono promosse a net del diagramma. Ogni net è rappresentata con un identificativo univoco (N1 , N2 , ...), utile per i successivi passi di matching terminale-net e di esportazione del grafo.

06_match_terminals_to_nets

Valutazione generale

Il passo 06 è uno dei punti in cui la pipeline batch v1 introduce una differenza metodologica importante rispetto al pilot.

In entrambe le versioni lo scopo dello script è lo stesso: partire dalle net costruite nel passo 05 e assegnare a ogni terminale una net finale, producendo così le connessioni terminale→net del diagramma.

Tuttavia, la logica usata per effettuare questo matching cambia in modo sostanziale:

- **pilot**: il match è basato soprattutto sulla prossimità locale del terminale a una net nel `label_map` ;
- **batch v1**: il match integra informazione proveniente dal passo 05, ricerca direzionale coerente con il lato del terminale, fallback multipli e una misura esplicita di affidabilità del collegamento.

Logica del pilot

Nel pilot il matching terminale → net funziona in modo relativamente semplice:

1. si legge la `label_map` prodotta dal passo 05;
2. per ogni terminale si guarda in una zona circolare attorno al punto `(x, y)` ;
3. se nella zona è presente una sola net, la si assegna direttamente;
4. se nella zona sono presenti più net, si sceglie quella con il pixel più vicino;
5. se non viene trovata alcuna net, il terminale resta `unmatched` .

Questa logica è implementata con:

- `get_candidate_labels_in_radius(...)`
- `nearest_label_by_pixel_distance(...)`
- `match_terminal_to_net(...)`

Nel pilot, quindi, il criterio fondamentale è la **vicinanza locale nel label map**, con un fallback a un raggio più grande (`FALLBACK_RADIUS`) se il primo tentativo fallisce.

Limiti emersi nel pilot

Questa soluzione funziona bene nei casi semplici, ma nel passaggio alla batch v1 sono emersi alcuni limiti:

- il terminale stimato non cade sempre esattamente sul pixel della net;
- una ricerca solo circolare non sfrutta la direzione del terminale (`left` , `right` , `top` , `bottom`);
- nei casi ambigui con più net vicine, la sola distanza locale può non essere sufficiente;
- classi come **Switch** e **Inductor** richiedono una ricerca più ampia o più permissiva;
- il pilot non distingue tra match forti e match deboli: restituisce solo `matched` o `unmatched` .

Per questo motivo il passo 06 è stato reso più robusto.

Modifiche introdotte nella batch v1

1. Integrazione dell'informazione prodotta dal passo 05

La batch v1 non tratta più il passo 06 come uno stadio completamente indipendente dal 05.

Oltre a leggere `nets` e `label_map` , utilizza anche l'informazione prodotta nel passo precedente per recuperare una **net preferita** per ciascun terminale.

Questa net preferita deriva dal `primary_label` calcolato nel 05, che viene poi convertito nella corrispondente `net_index` tramite la mappa `source_label -> net_index` .

Di conseguenza, il terminale non viene più matchato “alla cieca” sul label map, ma può partire da una preferenza locale già stimata in precedenza.

2. Passaggio da ricerca circolare semplice a ricerca multi-stage

Nel pilot il matching si basava su due soli tentativi:

- ricerca con `MATCH_RADIUS` ;
- fallback con `FALLBACK_RADIUS` .

Nella batch v1, invece, il matching segue un vero e proprio **piano di ricerca multi-stage**:

1. `directional_primary`
2. `circle_primary`

3. `directional_fallback`

4. `circle_fallback`

Questa sequenza è costruita da `build_search_plan(...)` .

Il vantaggio è che il matching non usa più una sola nozione di vicinanza, ma prova prima una ricerca **geometricamente coerente** con il terminale e solo dopo passa a ricerche più permissive.

3. Introduzione della ricerca direzionale coerente con `relative_position`

Questa è una delle differenze più importanti del passo 06.

Nel pilot la finestra di ricerca è sempre circolare e isotropa.

Nella batch v1, invece, il primo tentativo di match usa una finestra direzionale costruita in base al lato del terminale:

- `left / right` → ricerca prevalente orizzontale;
- `top / bottom` → ricerca prevalente verticale.

Questa logica è implementata tramite:

- `build_directional_rect(...)`
- `collect_labels_in_rect(...)`
- `run_search_stage(...)`

La conseguenza è che il passo 06 diventa più coerente con la geometria del terminale stimata nel passo 03 e con il modo in cui il passo 04 ha preservato localmente la connessione col wire.

4. Parametri di ricerca differenziati per classi difficili

Nel pilot i raggi di ricerca sono unici per tutte le classi.

Nella batch v1, invece, viene introdotto un insieme di override specifici per classi più difficili, raccolti in `CLASS_SEARCH_OVERRIDES` .

Tra queste classi rientrano:

- `Switch`
- `Inductor`
- `Meter`

- `Current_Source`
- `Voltage_Source`

Per queste classi vengono aumentati:

- la lunghezza della ricerca direzionale;
- l'ampiezza della finestra;
- il raggio della ricerca circolare;
- il raggio del fallback.

Questa modifica è importante perché riconosce che non tutti i componenti hanno la stessa facilità di matching e che alcuni simboli richiedono una ricerca più ampia o più tollerante.

5. Priorità alla net suggerita dal passo 05

Nel pilot, quando più net sono candidate, la scelta avviene semplicemente in base al pixel più vicino.

Nella batch v1, invece, se tra le net candidate compare la net preferita proveniente dal passo 05, questa viene privilegiata.

Solo se la preferred net non è disponibile si passa ai criteri standard:

- `single_candidate`
- `nearest_candidate`

Questa scelta è implementata in `choose_best_label(...)`.

Quindi il passo 06 non si limita più a risolvere un'ambiguità geometrica locale, ma sfrutta anche un **vincolo topologico suggerito dal passo precedente**.

6. Introduzione esplicita dello snap point

Nel pilot il match finale salva:

- la net scelta;
- la distanza del match;
- il raggio usato.

Nella batch v1 viene salvato anche il vero e proprio **snap_point**, cioè il punto del label map a cui il terminale è stato effettivamente agganciato.

Questa informazione è molto utile perché:

- rende il matching esplicito e verificabile;
- permette di visualizzare il collegamento reale tra terminale e net;
- sarà utile anche per il debug e per eventuali esportazioni future.

7. Arricchimento dello stato del match

Nel pilot gli stati principali erano:

- `matched_single`
- `matched_nearest`
- `unmatched`

Nella batch v1 gli stati diventano più informativi:

- `matched_preferred`
- `matched_single`
- `matched_nearest`
- `unmatched`

Questo permette di distinguere tra:

- match coerente con la preferred net del passo 05;
- match risolto per unicità locale;
- match risolto per prossimità tra più alternative;
- assenza di match.

8. Introduzione della confidence del match

Questa è la modifica più importante dal punto di vista interpretativo.

Nel pilot il risultato finale dice solo se il terminale è matchato oppure no.

Nella batch v1, invece, ogni match viene classificato con una confidenza:

- `high`
- `medium`
- `low`

- none

La classificazione dipende da:

- tipo di match (`matched_preferred` , ecc.);
- distanza di snap;
- utilizzo o meno di fallback;
- presenza o meno di preferred net;
- classe del componente.

Questa logica è implementata in `classify_match_confidence(...)` .

In questo modo la pipeline non si limita più a produrre un collegamento, ma ne stima anche la **qualità**.

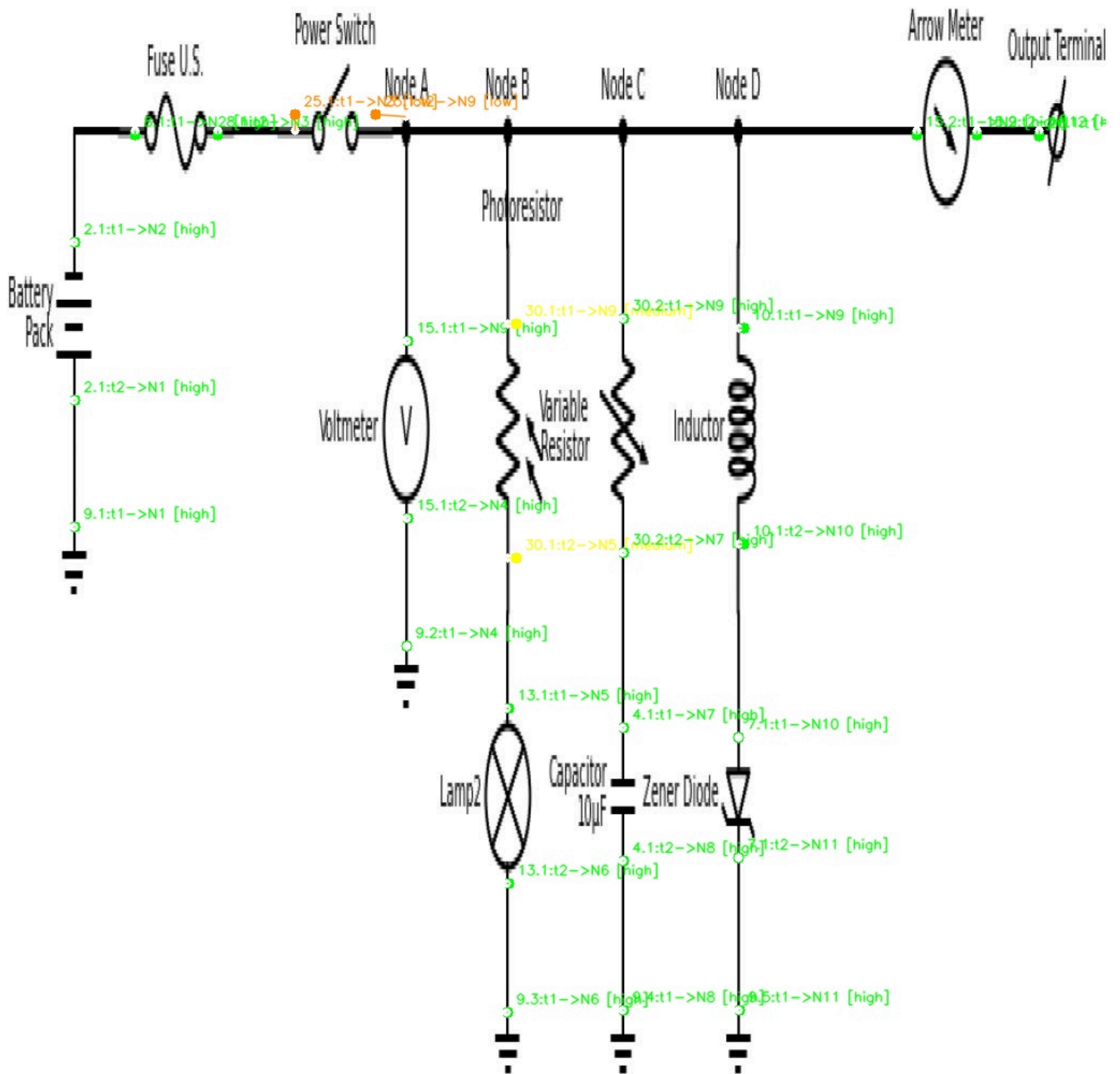


Figura 6.1 — Match terminale→net con confidenza bassa nel caso dello switch (e1e43 , 25.1).

L'immagine mostra un caso in cui il terminale viene comunque associato a una net, ma il collegamento è classificato come debole. La batch v1 non si limita a produrre il match finale: salva anche il punto di snap, la distanza di aggancio e una valutazione esplicita della confidence, così da evidenziare i casi sospetti.

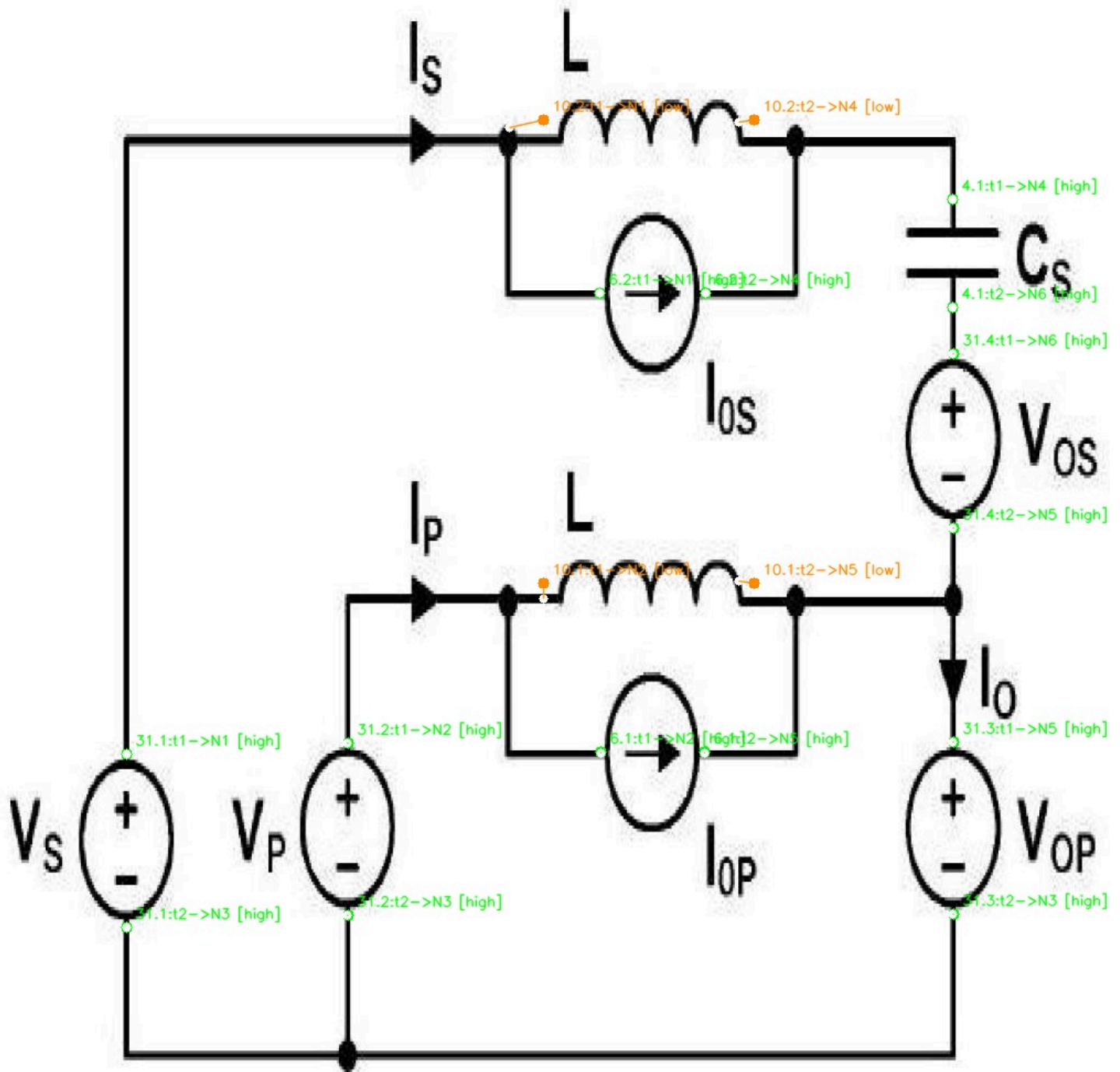


Figura 6.2 — Recupero di match difficili nel caso degli induttori (1855 , 10.1 e 10.2).

Questo esempio evidenzia il ruolo della ricerca multi-stage e dei parametri specifici per classi difficili. I terminali degli induttori vengono recuperati grazie a finestre di ricerca più ampie e fallback progressivi, ma il sistema mantiene traccia della qualità del risultato tramite confidence e warning.

9. Introduzione di warning espliciti per i match deboli

Oltre alla confidence, la batch v1 salva anche una lista di `match_warnings` , ad esempio:

- `no_preferred_net_from_05`
- `matched_without_preferred_label`
- `used_fallback_search`
- `used_circle_search`
- `large_snap_distance`
- `very_large_snap_distance`
- `sensitive_class_large_distance`

Questi warning trasformano il passo 06 in uno stadio molto più trasparente: il sistema non restituisce solo una connessione, ma spiega anche **perché quel match è considerato affidabile o sospetto**.

10. Introduzione del flag `is_suspicious_match`

Nel pilot non esisteva una nozione esplicita di “match sospetto”.

Nella batch v1, invece, ogni terminale può essere marcato con:

- `is_suspicious_match = true/false`

In pratica, i match a bassa confidenza vengono mantenuti nella pipeline, ma vengono anche etichettati come casi da trattare con maggiore cautela.

Questa scelta è particolarmente utile per non perdere collegamenti recuperati in casi difficili, pur mantenendo tracciabilità della loro affidabilità.

11. Arricchimento delle strutture di output

Nel pilot il passo 06 aggiorna:

- `components`
- `terminals`
- `connections`
- `terminal_net_matching`

Nella batch v1 queste strutture vengono notevolmente arricchite.

Per ogni terminale vengono salvati anche:

- `preferred_net_index_from_05`

- `preferred_net_id_from_05`
- `snap_point`
- `search_stage`
- `search_window`
- `search_kind`
- `match_confidence`
- `match_warnings`
- `is_suspicious_match`

Anche `connections` viene arricchito con:

- `component_class_name`
- `snap_point`
- `match_confidence`
- `match_warnings`
- `is_suspicious_match`

Infine, il blocco `terminal_net_matching` aggiunge:

- note descrittive;
- parametri base di ricerca;
- override di classe;
- regole della confidence;
- conteggi per `high`, `medium`, `low`, `none` ;
- numero e lista dei terminali sospetti.

12. Evoluzione del debug visivo

Nel pilot il debug overlay colorava:

- in verde i terminali matchati;
- in giallo i `matched_nearest` ;
- in rosso gli unmatched.

Nella batch v1 il debug viene ristrutturato per rappresentare la **qualità del match**:

- verde = `high`
- giallo = `medium`
- arancione = `low`

- `ROSSO = unmatched`

Inoltre, se disponibile, viene disegnato anche:

- `lo_snap_point`
- il segmento tra terminale e punto di snap

Questo rende molto più leggibile la differenza tra match sicuri e match recuperati in modo debole.

Sintesi della differenza concettuale

La differenza principale tra pilot e batch v1 nel passo 06 può essere riassunta così:

- **pilot:** il terminale viene assegnato alla net più vicina nel `label_map`, con una logica basata principalmente sulla prossimità locale;
- **batch v1:** il terminale viene assegnato alla net combinando informazione proveniente dal passo 05, ricerca direzionale coerente con il lato del terminale, fallback progressivi e una stima esplicita dell'affidabilità del match.

Di conseguenza, il passo 06 della batch v1 non è più soltanto uno stadio di assegnazione finale, ma diventa anche uno stadio di **valutazione della qualità delle connessioni**.

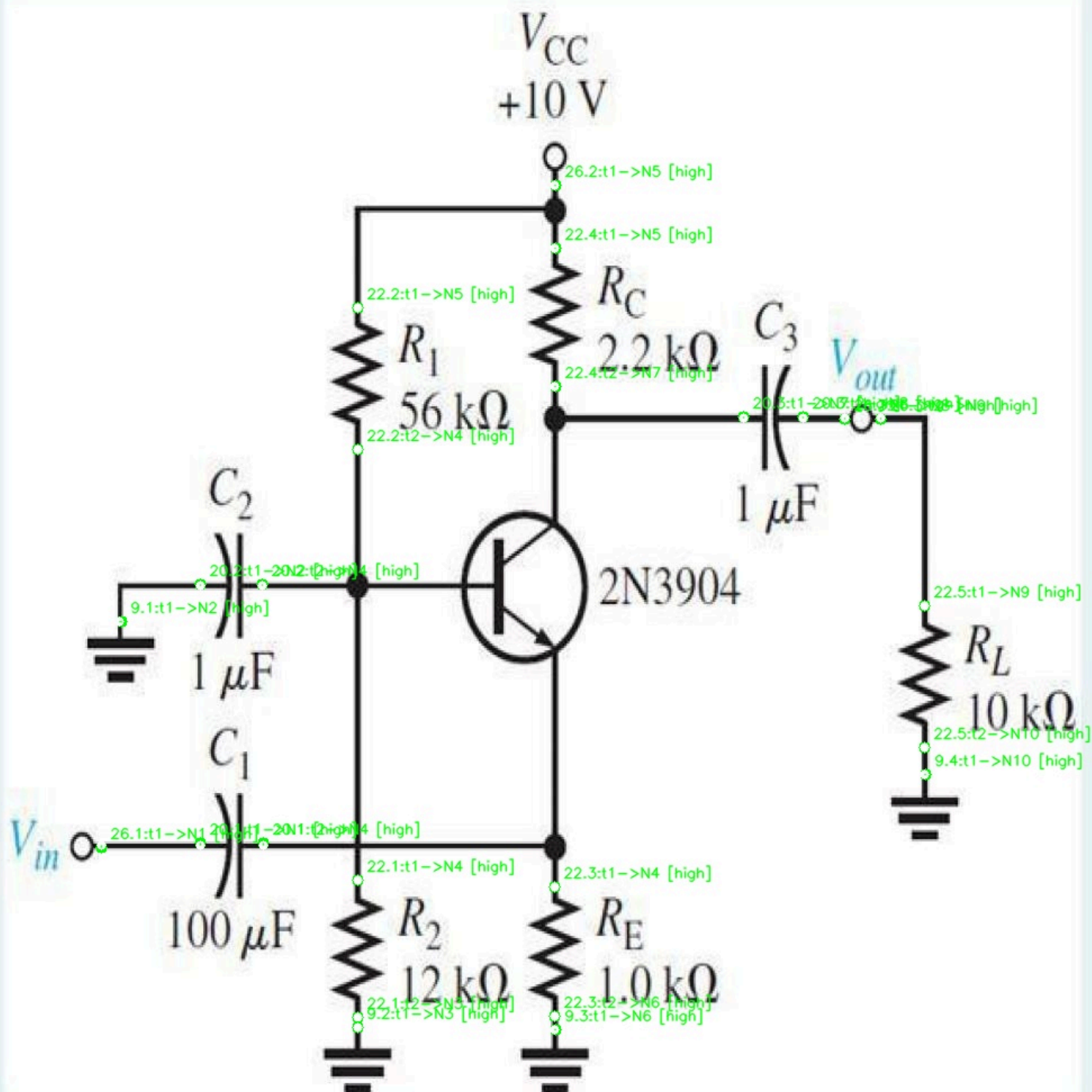


Figura 6.3 — Esempio di matching terminale→net ad alta confidenza.

In questo caso il collegamento tra terminali e net viene risolto in modo pulito, con distanza di snap ridotta e confidence alta. La figura è utile per confrontare il comportamento del sistema nei casi semplici con quello osservato nei casi più ambigui o sospetti.

07_export_graph

Valutazione generale

Il passo 07 ha lo scopo di trasformare l'output topologico del passo 06 in una rappresentazione esplicita di grafo, composta da:

- nodi di tipo `Diagram`
- nodi di tipo `Component`
- nodi di tipo `Terminal`
- nodi di tipo `Net`

e da archi di tipo:

- `HAS_COMPONENT`
- `HAS_NET`
- `HAS_TERMINAL`
- `CONNECTED_TO`

La logica concettuale di base è già presente nel pilot e rimane invariata nella batch v1: il passo 07 non costruisce nuova topologia, ma **esporta** in forma di grafo la struttura già prodotta dai passi precedenti. La differenza principale è che nella batch v1 l'esportazione diventa più robusta, più ricca di attributi e più adatta a un uso batch e a un successivo caricamento in un graph database.

Logica del pilot

Nel pilot lo script:

1. legge il JSON prodotto dal passo 06;
2. costruisce i nodi `Diagram`, `Component`, `Terminal`, `Net` ;
3. costruisce gli archi:
 - `Diagram -> HAS_COMPONENT -> Component`
 - `Diagram -> HAS_NET -> Net`
 - `Component -> HAS_TERMINAL -> Terminal`
 - `Terminal -> CONNECTED_TO -> Net`
4. salva:
 - un file `graph.json` per diagramma;
 - un file `nodes.csv` per diagramma;

- un file `edges.csv` per diagramma.

Questa struttura è già sufficiente a rappresentare il diagramma come grafo navigabile.

Modifiche introdotte nella batch v1

1. La struttura del grafo rimane invariata

La batch v1 mantiene invariata la struttura concettuale del grafo:

- stessi quattro tipi di nodo;
- stessi quattro tipi di relazione;
- stessa costruzione a partire dall'output del passo 06.

Questa continuità è importante: il passo 07 non cambia il modello logico del grafo, ma ne migliora l'esportazione.

2. Introduzione di ID univoci a livello batch

Questa è una delle differenze più importanti.

Nel pilot, gli identificativi dei nodi non includevano sempre il `diagram_id`. Per esempio:

- `component:{instance_id}`
- `terminal:{terminal_id}`
- `net:{net_id}`

Questo approccio funzionava bene per singoli diagrammi, ma in un batch multi-immagine poteva generare collisioni tra nodi con lo stesso nome provenienti da diagrammi diversi.

Nella batch v1 vengono introdotti ID univoci a livello batch:

- `diagram:{diagram_id}`
- `component:{diagram_id}:{instance_id}`
- `terminal:{diagram_id}:{terminal_id}`
- `net:{diagram_id}:{net_id}`

Questa modifica è fondamentale per rendere l'export compatibile con un graph database popolato con più diagrammi contemporaneamente.

3. Arricchimento dei nodi esportati

Nel pilot i nodi contenevano già gli attributi essenziali del diagramma, dei componenti, dei terminali e delle net.

Nella batch v1 i nodi vengono arricchiti con ulteriori proprietà provenienti dai passi precedenti. In particolare:

- il nodo `Diagram` include anche conteggi aggregati come:
 - `n_components`
 - `n_terminals_estimated`
 - `n_nets`
 - `n_connections`
 - `source_json_stage`
- il nodo `Component` include anche:
 - `estimated_connection_side`
 - `n_terminals`
- il nodo `Terminal` include anche:
 - `preferred_net_id_from_05`
 - `preferred_net_index_from_05`
 - `match_confidence`
 - `match_warnings`
 - `is_suspicious_match`
 - `search_stage`
 - `search_kind`
 - `search_window`
 - `snap_x`
 - `snap_y`
- il nodo `Net` include anche:
 - `source_label`
 - `connected_terminal_ids`

4. Arricchimento degli archi `CONNECTED_TO`

Nel pilot gli archi `CONNECTED_TO` contenevano principalmente:

- `terminal_id`

- `net_id`
- `match_status`
- `match_distance_px`

Nella batch v1 questi archi vengono estesi con:

- `component_class_name`
- `match_confidence`
- `match_warnings`
- `is_suspicious_match`
- `snap_point`

5. Introduzione di un summary del grafo più ricco

Nel pilot il `graph_summary` conteneva i conteggi principali di nodi e archi per diagramma:

- numero totale di nodi;
- numero totale di archi;
- numero di nodi per tipo;
- numero di archi per tipo.

Nella batch v1 il summary viene esteso con:

- `n_suspicious_terminal_matches`
- `terminal_match_confidence_counts` con conteggi per `high`, `medium`, `low`, `none`

Questa modifica è coerente con l'evoluzione del passo 06 e permette di avere, per ogni diagramma, una misura sintetica della qualità complessiva del matching terminale→net.

6. Introduzione di CSV aggregati a livello batch

Nel pilot vengono esportati solo:

- `graph_json` per diagramma;
- `nodes.csv` per diagramma;
- `edges.csv` per diagramma.

Nella batch v1, oltre a questi file, viene introdotta anche una cartella `combined_csv` con tre esportazioni batch:

- `all_nodes.csv`
- `all_edges.csv`
- `graph_summaries.csv`

Questa è una differenza pratica molto importante, perché rende molto più semplice:

- analizzare l'intero batch in modo tabellare;
- importare i dati in strumenti esterni;
- preparare il caricamento in un database a grafo.

7. Serializzazione esplicita di liste e dizionari nei CSV

Nel pilot la funzione `save_csv(...)` scriveva direttamente i dizionari nei CSV, assumendo valori semplici.

Nella batch v1 viene introdotta la funzione `jsonable(...)`, che converte automaticamente liste e dizionari in stringhe JSON prima della scrittura nel CSV.

Questo è importante perché molti attributi nuovi sono strutture non scalari, ad esempio:

- `match_warnings`
- `connected_terminal_ids`
- `search_window`
- `snap_point`
- `terminal_match_confidence_counts`

Di conseguenza, nei CSV della batch v1 questi campi compaiono come testo JSON serializzato all'interno della cella.

Cosa descrivono esattamente i CSV

1. `nodes_csv/<diagramma>_nodes.csv`

Questo file contiene **un record per ogni nodo del grafo relativo a un singolo diagramma**.

Le righe possono rappresentare quattro tipi diversi di nodo:

- Diagram
- Component
- Terminal
- Net

La colonna chiave per distinguerli è `node_type`.

In pratica, questo CSV descrive **l'insieme completo delle entità del diagramma**:

- il diagramma stesso;
- i componenti rilevati;
- i terminali stimati;
- le net costruite.

Le colonne presenti sono l'unione di tutti gli attributi usati dai diversi tipi di nodo. Questo significa che:

- alcune colonne sono valorizzate solo per certi tipi di nodo;
- per gli altri tipi rimangono vuote.

2. `edges_csv/<diagramma>_edges.csv`

Questo file contiene **un record per ogni arco del grafo relativo a un singolo diagramma**.

La colonna chiave è `relation_type`, che può assumere uno di questi valori:

- HAS_COMPONENT
- HAS_NET
- HAS_TERMINAL
- CONNECTED_TO

In pratica, questo CSV descrive **come le entità del diagramma sono collegate tra loro**:

- quali componenti appartengono al diagramma;
- quali net appartengono al diagramma;
- quali terminali appartengono a ciascun componente;
- a quale net è collegato ciascun terminale.

È quindi il file che rappresenta in modo esplicito la **topologia del grafo**.

3. combined_csv/all_nodes.csv

Questo file contiene **la concatenazione di tutti i nodi di tutti i diagrammi del batch**

Ogni riga rappresenta comunque un singolo nodo, ma qui i nodi di diagrammi diversi convivono nello stesso file.

Per questo motivo diventano essenziali:

- diagram_id
- node_id

Questo CSV è utile quando si vuole:

- analizzare tutto il batch insieme;
- esportare verso strumenti esterni;
- fare import massivo in un graph database.

4. combined_csv/all_edges.csv

Questo file contiene **la concatenazione di tutti gli archi di tutti i diagrammi del batch.**

Ogni riga rappresenta un singolo arco del grafo, con:

- nodo sorgente (source)
- nodo destinazione (target)
- tipo di relazione (relation_type)
- diagram_id

Questo è il CSV che descrive, a livello batch, **tutte le relazioni topologiche esportate.**

5. combined_csv/graph_summaries.csv

Questo file contiene **un record per diagramma**, non per nodo o per arco.

Ogni riga è un riepilogo statistico del grafo del singolo diagramma, con informazioni come:

- numero totale di nodi;
- numero totale di archi;
- numero di nodi Component , Terminal , Net ;

- numero di archi per tipo;
- numero di match sospetti;
- distribuzione della confidence dei terminali.

Quindi questo CSV non descrive il grafo in dettaglio, ma ne fornisce una **vista riassuntiva per immagine**.

Come leggere i CSV in modo corretto

Dal punto di vista interpretativo:

- `nodes.csv` descrive **quali entità esistono** nel grafo;
- `edges.csv` descrive **come queste entità sono collegate**;
- `all_nodes.csv` e `all_edges.csv` estendono la stessa logica all'intero batch;
- `graph_summaries.csv` descrive **le statistiche aggregate** del grafo per ciascun diagramma.

In altre parole:

- i CSV `nodes` e `edges` sono una rappresentazione tabellare del grafo;
- `graph_summaries.csv` è una rappresentazione tabellare delle sue metriche sintetiche.

Sintesi della differenza concettuale

La differenza principale tra pilot e batch v1 nel passo 07 può essere riassunta così:

- **pilot**: esportazione del grafo per singolo diagramma, con nodi e archi essenziali;
- **batch v1**: esportazione del grafo più ricca, con ID univoci a livello batch, attributi diagnostici aggiuntivi, summary della qualità dei match e CSV aggregati per l'intero batch.

Il passo 07 della batch v1 non modifica la topologia del grafo, ma ne migliora in modo sostanziale la **portabilità**, la **leggibilità** e l'uso successivo in analisi batch o in un graph database.